Práctica 4 de Lógica Computacional

Corte en Prolog

El corte y sus aplicaciones sobre listas

Erik Figueiral Alonso

erik.figueiral.alonso@uvigo.gal

19 de octubre de 2026

Índice

1 Control y aplicaciones

2 Ejercicios de examen

SWI-Prolog · Listas clásicas

Una salida del laberinto

Si solo queremos encontrar una salida, podemos dejar de buscar otras después del primer éxito.




```
una_salida(S) :- salida(S), !.
```

La decisión depende del objetivo: una salida o todas las salidas.

Apuntes, cap. 7, pp. 111–123 (PDF: desde p. 119).

Qué elimina el corte

```
p :- a, !, b.
p :- c.
```

Al llegar a !	Efecto en esta llamada a p
Alternativas pendientes de a	Se descartan
La otra cláusula de p	Se descarta
Alternativas que produzca b después	Pueden seguir explorándose

! tiene éxito. Si b falla después, no se vuelve a probar c.

SWI-Prolog, manual de !/0: swi-prolog.org/pldoc/man?predicate=!/0

Corte verde y corte rojo

CORTE VERDE

Evita trabajo o respuestas repetidas.
Al quitarlo, las respuestas distintas siguen siendo correctas y completas para el uso previsto.

```
miembro(1,[1,1]).
```

CORTE ROJO

La corrección depende del corte.
Al quitarlo, pueden aparecer respuestas incorrectas para la especificación.

```
int([a],[a],R).
```

El color depende del programa y de cómo se consulta, no del símbolo !.

Apuntes, cap. 7, §7.2, p. 123 impresa.

Pregunta: miembro/2 sin corte

Archivo control.pl

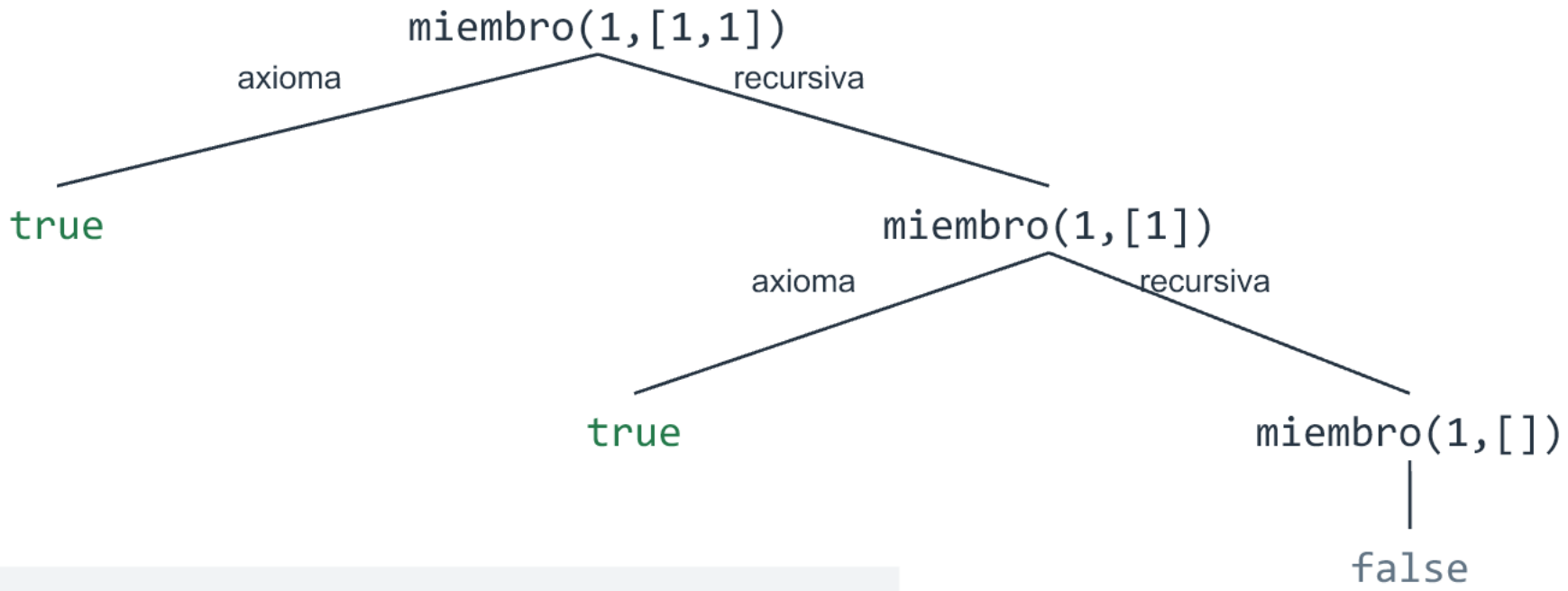
```
miembro(Car,[Car|_]).
miembro(X,[_|Cdr]) :- miembro(X,Cdr).
```

Consulta en SWI-Prolog

```
?- miembro(1,[1,1]).
```

¿Qué ocurre si pedimos todas las respuestas con ;?
Dibuja el árbol y distingue los éxitos del fallo final.

miembro/2: dos pruebas del mismo hecho



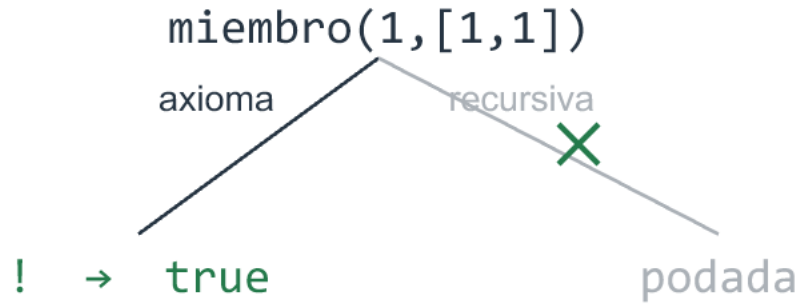

```
?- miembro(1,[1,1]).
true ; true ; false.
```

Tras el segundo true queda una alternativa: buscar en []. Esa rama falla.

miembro/2: detenerse en la coincidencia

Archivo control.pl

```
miembro(Car,[Car|_]) :- !.
miembro(X,[_|Cdr]) :-
    miembro(X,Cdr).
```




```
?- miembro(1,[1,1]).
true.
```

Verde en esta consulta: desaparece otra prueba del mismo hecho.

Una variable cambia la pregunta

Sin corte	Con corte en el axioma
?- miembro(E,[1,2]).	?- miembro(E,[1,2]).
E = 1 ; E = 2 ; false.	E = 1.

Con corte hay una sola respuesta: E = 1. Se acabó la búsqueda.
E = 2 también sería válida, pero se pierde. Esta versión comprueba pertenencia; no sirve para enumerar todos los elementos.

En las siguientes aplicaciones, el elemento y la lista estarán fijados.

Recordatorio: if_then_else/3

P, Q y R son objetivos

```
if_then_else(P,Q,_) :- call(P), !, call(Q).
if_then_else(_,_,R) :- call(R).
```

Si P tiene éxito, elegimos Q. Si P falla, elegimos R.
Una vez elegido Q, su fallo no permite volver a R.

Consulta	Respuesta
?- if_then_else(true,fail,true).	false.
?- if_then_else(false,fail,true).	true.

call(P) ejecuta el objetivo P. El corte va después de comprobar P.

Apuntes, cap. 7, ejemplo 7.1.5.

Ejercicio: intersección

Sintaxis y firma

```
int(A,B,R)      int/3
```

R contiene los elementos comunes de A y B.
A y B representan conjuntos mediante listas sin repetidos.

Consulta en SWI-Prolog	Resultado esperado
?- int([a,b],[b,c],R).	R = [b].
?- int([],[a],R).	R = [].
?- int([a],[b],R).	R = [].

Uso: A y B fijadas; R libre.

Solución: intersección

Archivo control.pl

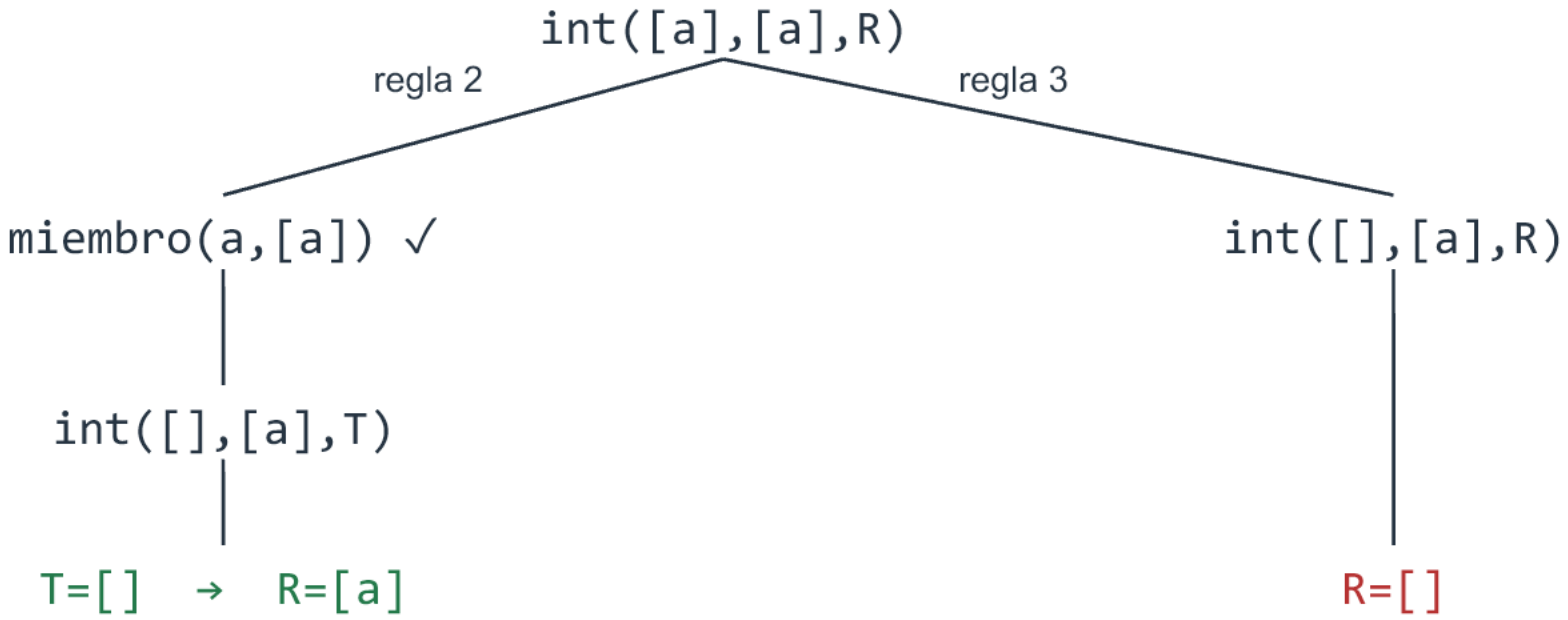
```
int([],_,[]).
int([Car|Cdr],C,[Car|T]) :-
    miembro(Car,C), !, int(Cdr,C,T).
int([_|Cdr],C,T) :-
    int(Cdr,C,T).
```

Si Car está en C, se incluye. En el otro caso, se descarta.
El corte rojo impide usar después la regla que lo descarta.

Consulta en SWI-Prolog	Respuesta
?- int([a,b],[b,c],R).	R = [b].

Intersección sin corte: una respuesta falsa

Suprimimos solo el corte de int/3. La llamada a miembro/2 se resume.

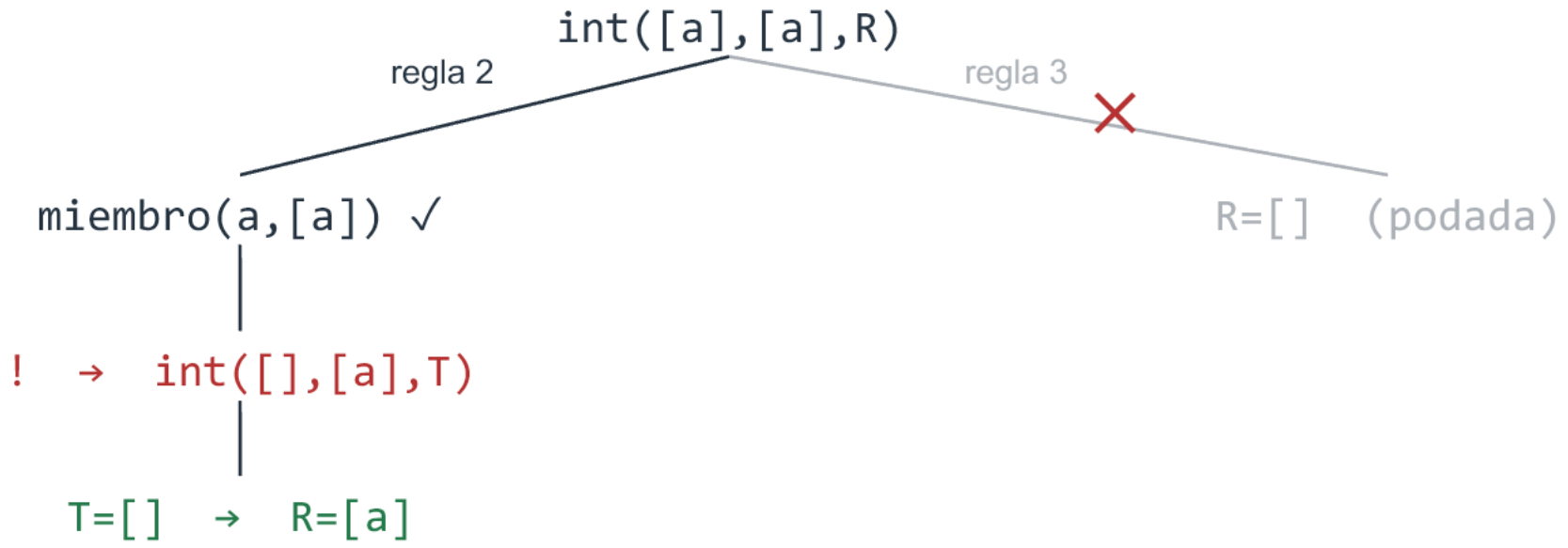



```
R = [a] ; R = [].
```

Árbol abreviado: miembro/2 se resuelve como una sola comprobación.

Intersección con corte: la decisión ya está tomada

```
miembro(Car,C), !, int(Cdr,C,T).
```



Primero sabemos que pertenece. Después cortamos la alternativa de excluirlo.

Ejercicio: unión

Sintaxis y firma

```
union(A,B,R)      union/3
```

R contiene los elementos de A o de B, sin repetirlos.
A y B son listas sin repetidos. Conservamos B al final.

Consulta en SWI-Prolog	Resultado esperado
?- union([a,b],[b,c],R).	R = [a,b,c].
?- union([],[a],R).	R = [a].
?- union([a],[a],R).	R = [a].

Uso: A y B fijadas; R libre.

Solución: unión

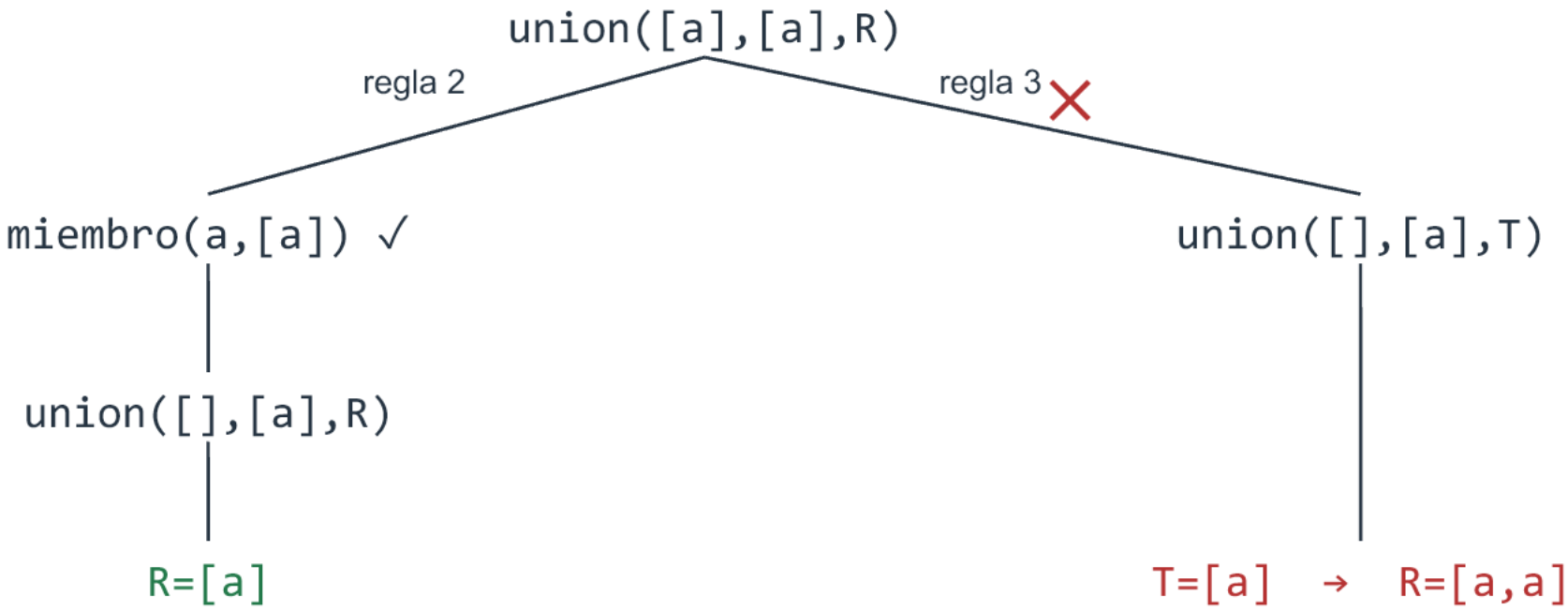
Archivo control.pl

```
union([],C,C).
union([Car|Cdr],C,T) :-
    miembro(Car,C), !, union(Cdr,C,T).
union([Car|Cdr],C,[Car|T]) :-
    union(Cdr,C,T).
```

Si Car ya está en C, no se añade otra vez.
La última regla lo añade solo cuando no pertenece a C.

Consulta en SWI-Prolog	Respuesta
?- union([a,b],[b,c],R).	R = [a,b,c].

Unión: el corte evita un duplicado incorrecto



Sin corte: ambas ramas.
Con corte: la rama derecha queda podada.

Los conjuntos y el modo de uso

Estas definiciones calculan el resultado a partir de dos conjuntos.
No eliminan automáticamente los repetidos de cualquier lista.

Fuera de la hipótesis sin repetidos	Qué sucede
?- int([a,a],[a],R).	R = [a,a].
?- union([a,a],[],R).	R = [a,a].

Para admitir repetidos habría que comprobar además
si el elemento ya aparece en el resultado y distinguir ese caso.

En esta práctica: listas sin repetidos y resultado inicialmente libre.

Ejercicio: contigua/3 con corte

Sintaxis y firma

```
contigua(X,Y,L)      contigua/3
```

X e Y aparecen uno junto al otro, en cualquiera de los dos órdenes.
Comprobad la relación sin repetir respuestas.

Consulta en SWI-Prolog	Resultado esperado
?- contigua(1,2,[1,2,1,2]).	true.
?- contigua(1,2,[2,1,2,1]).	true.
?- contigua(1,3,[1,2,3]).	false.

X, Y y L fijados. Podéis reutilizar izqda_der/3.

Solución: contigua/3

Archivo control.pl

```
contigua(X,Y,L) :- izqda_der(X,Y,L), !.
contigua(X,Y,L) :- izqda_der(Y,X,L), !.
```

Cada corte detiene la búsqueda al encontrar una pareja válida.
El segundo también evita repeticiones en el orden inverso.

Consulta en SWI-Prolog	Respuesta
?- contigua(1,2,[2,1,2,1]).	true.

Ejercicio: ady/3

Sintaxis y firma

```
ady(X,Y,L)      ady/3
```

Misma relación que contigua/3. Implementadla directamente sobre la lista, sin llamar a izqda_der/3 ni a contigua/3.

Consulta en SWI-Prolog	Resultado esperado
?- ady(1,2,[3,2,1,2]).	true.
?- ady(1,3,[1,2,3]).	false.
?- ady(1,2,[]).	false.

X, Y y L fijados. Una sola respuesta afirmativa.

Solución: ady/3

Archivo control.pl

```
ady(X,Y,[X,Y|_]) :- !.
ady(X,Y,[Y,X|_]) :- !.
ady(X,Y,[_|Cdr]) :- ady(X,Y,Cdr).
```

Los dos primeros casos reconocen vecinos.
La recursión avanza una posición hasta encontrarlos.

Consulta en SWI-Prolog	Respuesta
?- ady(1,2,[3,2,1,2]).	true.
?- ady(1,3,[1,2,3]).	false.

Comparaciones y unificación

Operador	Significado	Ejemplo
< y >	Comparación numérica estricta	2 < 5
=< y >=	Comparación numérica no estricta	3 =< 3
=	Unificación de términos	X = 3
=:=	Igualdad de valores numéricos	2 =:= 1+1

En Prolog se escribe =<. El operador = no evalúa expresiones aritméticas.

SWI-Prolog: manual, Arithmetic y Comparison and Unification of Terms.

X < 2 no genera valores para X

Consulta en SWI-Prolog	Resultado
?- X < 2.	Error de instanciación
?- X = 1, X < 2.	X = 1.
?- X = 1+1.	X = 1+1.
?- 2 = 1+1.	false.
?- 2 =:= 1+1.	true.

Las comparaciones aritméticas calculan y comparan valores.
Sus variables deben tener un valor numérico al ejecutarse.

Recordatorio: ¿qué era Quicksort?

Quicksort ordena una lista mediante «divide y vencerás».
Elegimos un pivote, separamos el resto y ordenamos las dos partes recursivamente. Al reunir izquierda, pivote y derecha, queda ordenada.

```
[3,1,4,2]  →  [1,2,3,4]     Pivote: 3
```

Menores o iguales que 3	Pivote	Mayores que 3
[1,2]	3	[4]

Primero definiremos partir/4 para separar la lista.
Después construiremos quicksort/2 y reuniremos el resultado con concat/3.

Ejercicio: partir/4

Sintaxis y firma

```
partir(Pivote,Lista,MenoresOIguales,Mayores)
```

MenoresOIguales contiene los elementos =< Pivote.
Mayores contiene los elementos > Pivote. Se conserva su orden.

Consulta en SWI-Prolog	Resultado esperado
?- partir(3,[1,4,3,2],I,D).	I = [1,3,2], D = [4].
?- partir(3,[],I,D).	I = [], D = [].

partir/4 · Pivote y Lista numéricos y fijados; I y D libres.

Solución: partir/4

Archivo control.pl

```
partir(_,[],[],[]).
partir(P,[Car|Cdr],[Car|I],D) :-
    Car =< P, !, partir(P,Cdr,I,D).
partir(P,[Car|Cdr],I,[Car|D]) :-
    partir(P,Cdr,I,D).
```

Lista vacía: ambas partes vacías. Si Car =< P, se coloca en I.
Si no se cumple, Car > P: la última regla lo coloca en D.

Consulta en SWI-Prolog	Respuesta
?- partir(3,[1,4,3,2],I,D).	I = [1,3,2], D = [4].

partir/4: un ejemplo paso a paso

```
?- partir(3,[1,4,3,2],I,D).
```

Cabeza	Comparación con 3	Dónde se coloca
1	1 =< 3	Izquierda
4	4 > 3	Derecha
3	3 =< 3	Izquierda
2	2 =< 3	Izquierda
[]	Caso base	I = [], D = []

```
I = [1,3,2], D = [4].
```

Ejercicio: quicksort/2

Sintaxis y firma

```
quicksort(Lista,Ordenada)      quicksort/2
```

Ordenada contiene los números de Lista en orden no decreciente.
Deben conservarse las repeticiones.

Consulta en SWI-Prolog	Resultado esperado
?- quicksort([3,1,4,2],R).	R = [1,2,3,4].
?- quicksort([3,1,3],R).	R = [1,3,3].
?- quicksort([],R).	R = [].

Lista numérica y fijada; R libre. Disponibles: partir/4 y concat/3.

Solución: quicksort/2

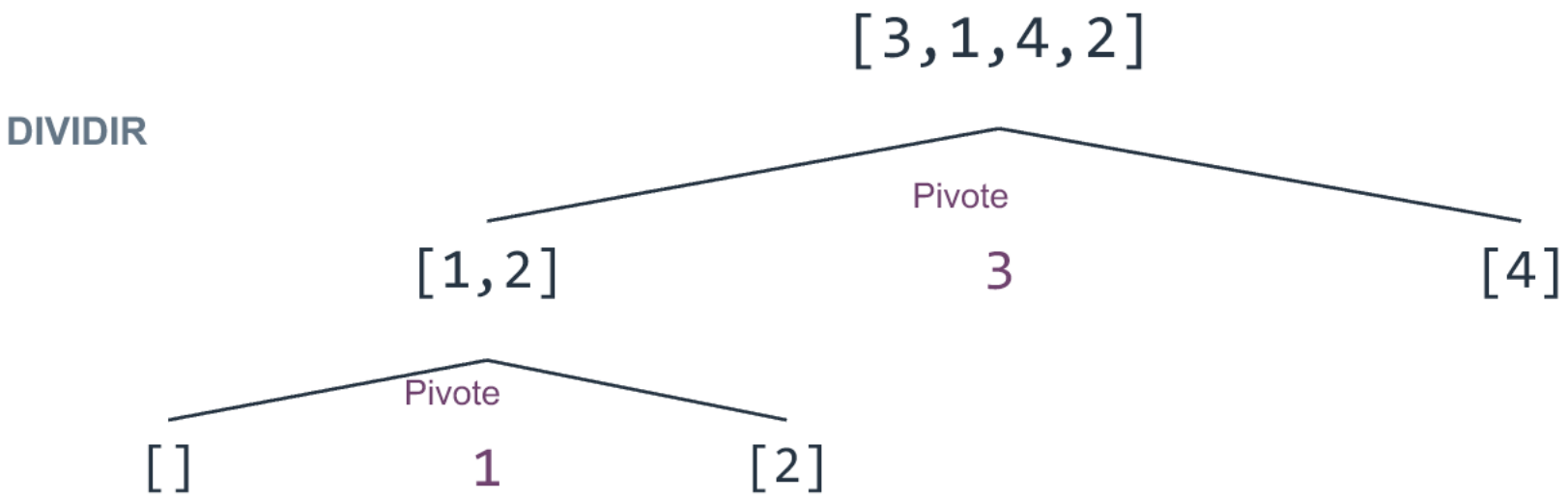
Archivo control.pl

```
quicksort([],[]).
quicksort([Car|Cdr],R) :-
    partir(Car,Cdr,I,D),
    quicksort(I,IO),
    quicksort(D,DO),
    concat(IO,[Car|DO],R).
```

IO y DO son las dos partes ya ordenadas.
El pivote se coloca entre ellas.

Consulta en SWI-Prolog	Respuesta
?- quicksort([3,1,4,2],R).	R = [1,2,3,4].

Quicksort: separar, ordenar y concatenar



REUNIR `[] + [1] + [2] → [1,2]` `[4]`

```
[1,2] + [3] + [4]  →  [1,2,3,4]
```

\+ representa concatenar listas; no es el operador aritmético de Prolog.

Tres ejercicios del primer examen práctico

Curso 2018–19

1 superiores/3

2 listar_hasta/3

3 creciente/1

Trabajad a partir del enunciado y de las consultas de prueba.
Las tres soluciones están al final del bloque.

Enunciados proporcionados: descripcion-LC1-LC2, LC3-LC4 y LC5-LC6.

Examen 1: superiores/3

Sintaxis y firma

```
superiores(Elem,Lista,Resultado)      superiores/3
```

Resultado contiene todos los elementos de Lista superiores a Elem.
Los datos son naturales. El orden del resultado es indiferente.

Consulta en SWI-Prolog	Resultado esperado
?- superiores(5,[4,8,2,3,6],X).	X = [8,6].
?- superiores(5,[],X).	X = [].
?- superiores(5,[4,2],X).	X = [].

El enunciado original exige que la primera respuesta sea correcta.

Examen 2: listar_hasta/3

Sintaxis y firma

```
listar_hasta(Lista,Elem,Respuesta)      listar_hasta/3
```

Respuesta contiene los elementos anteriores a la primera aparición de Elem. Esa aparición queda excluida.

Consulta en SWI-Prolog	Resultado esperado
?- listar_hasta([1,2,3,4],3,X).	X = [1,2].
?- listar_hasta([1,2,3,4],1,X).	X = [].
?- listar_hasta([],4,X).	X = [].

El enunciado original exige que la primera respuesta sea correcta.

Examen 3: creciente/1

Sintaxis y firma

```
creciente(Lista)      creciente/1
```

La lista de naturales es estrictamente creciente.
Según este enunciado, [] no se considera creciente.

Consulta en SWI-Prolog	Resultado esperado
?- creciente([2,5,8,9]).	true.
?- creciente([2,5,1,8,9]).	false.
?- creciente([4]).	true.
?- creciente([]).	false.

Podéis usar el operador <.

Pruebas adicionales de los ejercicios

Consulta	Resultado esperado
?- superiores(5,[5,8,8],R).	R = [8,8].
?- listar_hasta([1,2,1,3],1,R).	R = [].
?- listar_hasta([1,2],9,R).	R = [1,2].
?- creciente([2,2,3]).	false.

El caso en que Elem no aparece sigue el comportamiento del código de referencia: se devuelve toda la lista.

Pruebas añadidas. Caso ausente en listar_hasta/3: código de referencia.

Solución de examen: superiores/3

Archivo control.pl

```
superiores(_,[],[]).
superiores(Elem,[Car|Cdr],[Car|R]) :-
    Elem < Car, !, superiores(Elem,Cdr,R).
superiores(Elem,[_|Cdr],R) :-
    superiores(Elem,Cdr,R).
```

El corte rojo evita descartar un elemento que debe aparecer.
Los elementos iguales a Elem quedan fuera.

Consulta en SWI-Prolog	Respuesta
?- superiores(5,[4,8,2,3,6],R).	R = [8,6].

Solución de examen: listar_hasta/3

Archivo control.pl

```
listar_hasta([],_,[]).
listar_hasta([Elem|_],Elem,[]) :- !.
listar_hasta([Car|Cdr],Elem,[Car|R]) :-
    listar_hasta(Cdr,Elem,R).
```

Al encontrar Elem, el resultado restante es [].
El corte rojo impide continuar por detrás de esa aparición.

Consulta en SWI-Prolog	Respuesta
?- listar_hasta([1,2,3,4],3,R).	R = [1,2].

Solución de examen: creciente/1

Archivo control.pl

```
creciente([_]).
creciente([Car,Cadr|Cddr]) :-
    Car < Cadr, creciente([Cadr|Cddr]).
```

Se comparan vecinos y se continúa desde el segundo.
No hay cláusula para []. Aquí no hace falta ningún corte.

Consulta en SWI-Prolog	Respuesta
?- creciente([2,5,8,9]).	true.
?- creciente([]).	false.